Tutorial 5

1. *Given a BST T and a key k, the task is to delete all keys $<$ k from T.*

   (a) Write pseudocode to do this.

   (b) How much time does your algorithm take?

   (c) What is the structure of the tree left behind?

   (d) What is its root?

Solution:

(a) The code (See Algorithm 1) provided deletes all keys $< k$ from T and returns the root of the modified BST.

(b) The complexity of the procedure is $\mathcal{O}(h)$, h being the height of T, as we are visiting each level atmost once (worst case being reaching a leaf node).

(c) The structure of the tree left behind is still a BST.

(d) If the previous root was greater than or equal to k, then the new root is the same as the previous root. Otherwise, the new root is different.

Algorithm 1: Deleting all keys $< k$

```
Function DeleteKeys(curr, k):
    if curr = NULL then
        return NULL
    end
    if curr → val ≥ k then
        (curr → left) ← DeleteKeys((curr → left), k)
        return curr
    end
    else
        return DeleteKeys((curr → right), k)
    end
end
// Call DeleteKeys(root, k) in the main function
```

2. *Let $H(n)$ be the expected height of the tree obtained by inserting a random permutation of [n]. Write the recurrence relation for $H(n)$.*

Solution: The height of a binary tree equals one plus maximum of that of left and right subtrees. In a random permutation of $[n]$, any element (say k) can be the root of the subtree with probability $1/n$. The left subtree will contain $k-1$ nodes and the right one $n-k$.

Let X_n be the random variable representing the height of a n-node BST. Also, $H(n) = E[X_n]$. Then the recurrence relation is given as :-

$$\forall n \in \mathbb{N},\ H(n) = E[X_n] = 1 + \frac{1}{n}(\sum_{i=1}^{n} E[\max(X_{i-1}, X_{n-i})])$$
$$\leq 1 + \frac{1}{n}(\sum_{i=1}^{n} E[X_{i-1} + X_{n-i}])$$
$$= 1 + \frac{1}{n}(\sum_{i=1}^{n} E[X_{i-1}] + \sum_{i=1}^{n} E[X_{n-i}])$$
$$= 1 + \frac{2}{n}(\sum_{i=1}^{n-1} E[X_i]) = 1 + \frac{2}{n}(\sum_{i=1}^{n-1} H_i)$$

Base Case: $H(0) = 0$

Solving the recurrence gives $H(n)$ as $\mathcal{O}(\log n)$. You can refer this for a rough idea of its solution.

3. *Prove that after rotation the resulting tree is a binary search tree.*

Solution: See Figure 1. Suppose we go from left to right.

A becomes the left child of B and T_2 becomes the right child of A. $A \to val \leq B \to val$ as B was initially the right child of A (definition of BST) and $T_2 \to val \geq A \to val$ as T_2 was originally in the right subtree of A. T_1 is a BST, so is T_2, hence A's subtree is a BST too. Hence the new structure follows the rules of a BST.

Similarly we can prove for right to left in the figure.

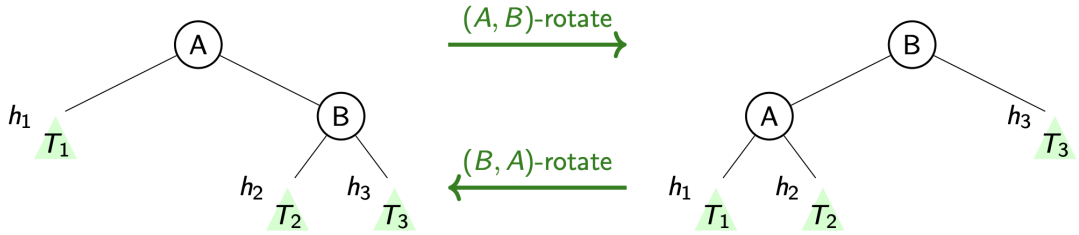

Figure 1: Rotation

4. *Insert sorted numbers 1,2,3,..., 10 to an empty red-black tree. Show all intermediate red-black trees.*

Solution: See Figure 2 below.

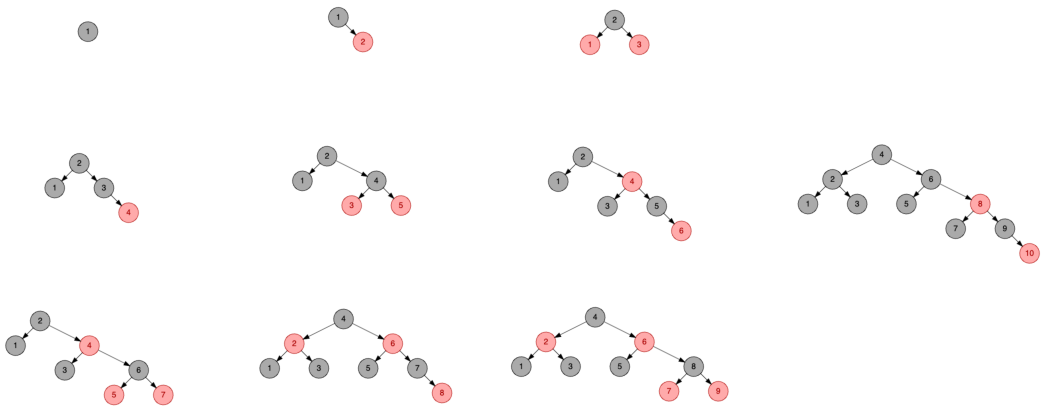

Figure 2: RBT Tree Insertions

5. *Consider the tree below. Can it be coloured and turned into a red-black tree? If we wish to store the set* $1, \ldots, 22$*, label each node with the correct number. Now add* 23 *to the set and then delete* 1*. Also, do the same in the reverse order. Are the answers the same? When will the answers be the same?*

tutorial˙5˙4.png

Solution: If we color this RB tree, Black height of this tree must be 3. Why?? (Hint: Argue in terms of property of RB tree) Colour the nodes H, E, O, C, N, V, K and M red; the rest should be black. This is a red-black tree with a black height of 3. Using the inorder traversal of the tree,
$A = 8, B = 3, C = 14, D = 2, E = 6, F = 12, G = 18, H = 1, I = 5, J = 7, K = 10$
$L = 13, M = 16, N = 20, O = 4, P = 9, Q = 11, R = 15, S = 17, T = 19, U = 21, V = 22$

Insertion Followed by Deletion
The insertion is very straightforward as 23 gets added as the right child of V = 22, let's call it W. Red black tree condition gets violated as V and W are of same color now. So, according to case 3, apply a left rotation such that after rotation N.right=V, V.left=U and V.right = W. H=1 is a red color leaf, so we can simply delete it without any trouble of black height violation.

Deletion Followed by Insertion
In this case, our answer will be the same as the one above. Since, deletion of H doesn't change anything significant for the right tree and we can simply insert 23 as described above.
Difference can be in cases where there is upward propagation of violation while dealing with violation after insertion and deletion. Since, upward propagation checks and affect different parts and colors of the tree

6. *(a) Give running time complexities of delete, insert, and search in red-black tree.*

   (b) What are the advantages of red-black tree as compare to Hash table, where every thing is constant time?

Solution: (a) $\mathcal{O}(\log n)$, n being the number of nodes of RBT. (Refer slides to know why).

(b) These are the following advantages:-

- Red-black trees maintain elements in a sorted order, which is useful for operations that require order (e.g., finding the minimum/maximum, range queries). Hash tables, on the other hand, do not maintain any order among elements.
- The performance of a red-black tree is predictable and does not depend some factor like hash table does on the quality of the hash function or the distribution of data.
- RBT has no collisions like a Hash Table.

7. *Suppose we use a hash function h to hash n distinct keys into an array T of length m. Assuming simple uniform hashing, what is the expected number of collisions? More precisely, what is the expected cardinality of* $\{\{k,l\} : k \neq l \text{ and } h(k) = h(l)\}$*?*

Solution: Let's define a binary random variable X_{ij}:-

$$X_{ij} = \begin{cases} 1 & \text{if there is a collision between the } i^{th} \text{ and } j^{th} \text{ keys during insertion,} \\ 0 & \text{otherwise.} \end{cases}$$

$E[X_{ij}] = 1/m$ as the probability that the key that is inserted later gets the same slot as the one inserted earlier is $1/m$. Our hashing function is **uniform**, i.e. probability of getting any slot is equal and hence is 1/(number of slots).

Let X be the random variable returning the number of collisions during insertion of the n keys, then,

$$X = \sum_{i=1}^{n} \sum_{j=i+1}^{n} X_{ij}$$

This is because the RHS represents the sum over all possible distinct unordered pairs $\{\{k,l\}, k \neq l\}$ seeing if there is a collision within each pair or not. Hence,

$$\begin{aligned} E[X] &= E[\sum_{i=1}^{n} \sum_{j=i+1}^{n} X_{ij}] \\ &= \sum_{i=1}^{n} \sum_{j=i+1}^{n} E[X_{ij}] \text{ (Due to linearity of the Expectation operator)} \\ &= \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{1}{m} \text{ (Explained above)} \\ &= \frac{\binom{n}{2}}{m} = \frac{n(n-1)}{2m} \text{ (There are } \binom{n}{2} \text{ total unordered pairs)} \end{aligned}$$

Hence the expected number of collisions is $\frac{n(n-1)}{2m}$.